

AIPLA — Internal Hosting Requirements

Resources and questions for University of Copenhagen IT

AIPLA project team

2026-08-27

Table of contents

1. Purpose	2
2. What the application is (in brief)	2
3. Data residency and privacy (why hosting at UCPH is attractive)	3
4. Hosting options	3
4b. Relationship to the KU-wide AI platform	4
Option 1 — AIPLA as a tenant	4
Option 2 — AIPLA as a template for the KU platform	4
Option 3 — parallel systems	6
What we suggest	6
5. Resource list (what UCPH would provide)	6
6. AI models and GPU sizing (the main open decision)	7
Responsiveness and capacity	8
7. Questions for UCPH IT	9
A. About the KU platform (new — see section 4b)	9
B. About hosting AIPLA itself	10
8. What we can contribute	10

i Draft for review — revised August 2026

Prepared for the University of Copenhagen IT department. Alongside it we provide infrastructure-as-code (Terraform) and operational runbooks so your team can stand the system up against the resources below, or use them as a costing reference for your own deployment.

What changed in this revision. The first version (June 2026) asked one question: *can you host this?* Since then, two things have altered the picture. KU has committed **110 million DKK over three years from 1 September 2026** to integrating AI across research, teaching and administration, with a new AI office, a cross-faculty taskforce, AI-labs and a significant strengthening of KU IT — and KU IT is already building internal infrastructure with locally hosted models, aiming at a KU-wide AI platform where staff and students sign in with their KU credentials and choose a model appropriate to the task. Separately, AIPLA's technical lead is engaged with the project to at least April 2027, so this document is no longer a farewell handover; it is the opening of a working conversation.

That changes the question. **Section 4b** is new and is the part we would most like your response to: it sets out three possible relationships between AIPLA and the platform KU IT is building — AIPLA as a *tenant* of it, AIPLA as a *template* for it, or the two built in parallel. We have a view, but it is yours to decide.

1. Purpose

AIPLA (AI in Physics Learning and Assessment) is a three-year research programme at the Center for Digital Education studying how generative AI can be used in upper-secondary (stx) physics teaching and assessment. Its software platform has been running since May 2026, has been in live use with Danish gymnasium teachers since August 2026, and currently runs on Google Cloud in the EU (Finland region).

This document has two purposes:

1. **To specify what the application needs to run inside University of Copenhagen infrastructure** — fully on-premises or hybrid — so your team can assess feasibility and estimate effort. Sections 2–3 and 5–6.
2. **To open a decision about the relationship between AIPLA and the KU-wide AI platform KU IT is now building.** Section 4b. These are two efforts pointed at overlapping problems, and the cheapest outcome for both is almost certainly not to build them twice.

It ends with a short set of questions we need answered to firm up either plan.

The application is built on portable, open standards — Linux containers, PostgreSQL, S3-compatible object storage, OpenID Connect single sign-on, and open-source model inference. Migrating it is therefore a matter of providing equivalent infrastructure, **not** rewriting the software.

2. What the application is (in brief)

Component	What it does	Standard technology
Web application	Student chat interface + teacher dashboard	Linux containers (Next.js + Python)
Database	Application data + document-search index	PostgreSQL (with the pgvector extension)
Object storage	Uploaded documents, optional lesson audio, caches	S3-compatible (e.g. MinIO) or NFS
AI model inference	The language models behind the tutor	Open-weight models served on GPUs (or cloud API)
Authentication	Teacher sign-in; anonymous student join	Institutional SSO (OIDC/SAML) + anonymous codes
Observability	Logs, metrics, traces (kept internal)	OpenTelemetry → Grafana/Prometheus or ELK

The web application and database are modest in size; the **GPU for model inference is the main new hardware consideration**, and is covered in sections 5–6.

3. Data residency and privacy (why hosting at UCPH is attractive)

This is likely the strongest reason to host internally:

- **No personally identifying student data is collected — by design.** Students join a lesson by entering an anonymous group code on a shared device. The system stores nothing that identifies an individual student: no names, no emails, no device identifiers. The “personal data” category is empty by construction, which substantially simplifies data-protection review.
- **Teachers** sign in with institutional credentials (single sign-on).
- **Today** all data is held in the EU (Google Cloud, Finland). **Hosting at UCPH** would place all data — including any optional, consent-gated lesson recordings — entirely on university infrastructure: the strongest possible data-sovereignty posture.

4. Hosting options

There is a spectrum between today’s full-cloud setup and a fully on-premises deployment. UCPH does not have to choose the extreme.

	A. Full cloud (today)	B. Inference on-prem	C. App + data on-prem (<i>recommended</i>)	D. Fully on-prem
AI models	Cloud API	UCPH GPUs serve most; cloud for overflow	UCPH GPUs serve most; cloud API only for the hardest tasks	All on UCPH GPUs
App + database + storage	Cloud (EU)	Cloud (EU)	UCPH	UCPH
Student data leaves UCPH?	Stays in EU cloud	Stays in EU cloud	No (except a few model prompts)	No
GPU hardware needed	None	High	Medium (single high-end GPU)	High (GPU cluster)
Effort	—	Low–medium	Medium	High

Our recommendation is C: the application and all student data live at UCPH, while a small set of the most demanding AI tasks may still use an EU cloud model API until on-premises models are proven equal. **B** is a sensible stepping stone if GPUs arrive before the rest of the migration. **D** is the end state once a GPU cluster is in place.

4b. Relationship to the KU-wide AI platform

Section 4 asks *where AIPLA's data and models sit*. This section asks a different question: **what relationship should AIPLA have to the general AI platform KU IT is building?**

We understand that KU IT is standing up internal infrastructure with locally hosted models, with the goal of a platform where staff and students sign in with their KU credentials and select a model suited to the task at hand. AIPLA already does a version of this for one discipline. There are three ways those two facts can be resolved, and the choice is KU's.

Option 1 — AIPLA as a tenant

AIPLA keeps its own application layer and consumes the KU platform for the two things that are genuinely institutional: **model inference** and **teacher sign-on**.

- **What KU provides:** an inference endpoint (an OpenAI-compatible HTTP API is the common denominator, and is what both vLLM and Ollama serve), and an OIDC client registration.
- **What changes in AIPLA:** the model router gains one more provider entry; teacher authentication points at the KU identity provider. Both are configuration-scale changes rather than rewrites — the router already dispatches across three provider families — and the student path needs nothing from KU at all, since students are anonymous by design.
- **Effort:** low. Days rather than weeks.
- **What it does not solve:** the two systems still duplicate everything above the model call — spend governance, logging, document handling, and the per-subject configuration surface.

This is the sensible first step whatever else is decided, and we would like to do it regardless of the answer to the rest of this section.

Option 2 — AIPLA as a template for the KU platform

The KU platform takes AIPLA's application layer as its starting point, and AIPLA becomes the physics instance of it rather than a separate system.

This is worth considering for one specific reason: **AIPLA is already a fork of a general-purpose platform, not a bespoke physics application**. It was built by forking a domain-neutral template and adding a physics configuration layer on top. The separation between “the platform” and “one subject's use of it” is not a retrospective claim — it is how the code is organised, and the existence of the fork is evidence that it separates cleanly.

Concretely, the following already exist and are in live use. They are the parts a general institutional platform needs, rather than the parts specific to physics:

Capability	State today
Model routing across providers	Dispatches across Gemini (Vertex), Claude (Vertex), and any OpenAI-compatible endpoint. Which model serves which task is a configuration field, not code.

Capability	State today
Per-task, per-user spend governance	A register of who may spend, monthly caps in real currency, tiers, expiry dates, and per-task cost multipliers. Enforced at request time rather than reconciled after the fact. This is usually the hardest part of an institutional AI platform to retrofit.
Two identity modes	Institutional single sign-on for staff, plus a fully anonymous join path for people who must not be identified. Most platforms have only the first.
Configuration by subject, not by code	Each use case is a plain-text configuration document — instructions, tools, model tier, budget policy — versioned in a repository and deployed automatically. A department can author one without a developer.
Document handling	Parsing across 13 deterministic formats plus AI extraction, and retrieval-augmented search over an institutional document corpus with per-corpus access control.
Interactive content	Sandboxed browser applications (simulations, interactive exercises) inside an enforced security envelope — content-security policy, iframe sandboxing, size limits, no external network access.
Observability and research data	OpenTelemetry traces, structured logs, and an anonymised interaction log in a queryable analytics store — designed for research use from the start rather than added afterwards.
Open protocols	Speaks MCP, A2A, AG-UI and A2UI, so it interoperates with tools built elsewhere instead of requiring everything to be built inside it.
Infrastructure as code	Terraform across three environments with artefact-based promotion, plus a local mode that runs the entire system with no cloud dependency.

There is also a governance argument, which we raise tentatively because it is not ours to make. If the intention is that **each subject works out for itself what AI means for its teaching**, then a platform whose unit of configuration is *a subject's own plain-text document* is the technical expression of that policy, where a platform that centralises those decisions is not. AIPLA is a worked example of one subject doing exactly this, with a live teacher pilot behind it.

What we would say against this option, plainly:

- **Scale is unproven.** AIPLA is sized and tested for roughly ten teachers and their classes. Nothing in the architecture forbids institutional scale, but nothing in its history demonstrates

it either. A KU-wide platform is a different engineering problem and we would not want that difference glossed over.

- **It carries migration debt.** Three components were chosen for speed of delivery and are specific to the current cloud: the application database, the retrieval store, and the session store. Our internal migration notes cost this honestly. That debt is tolerable in a research prototype and less tolerable in a platform other people build on.
- **It is a research prototype, not a supported product.** It has one principal developer. Adopting it means either resourcing it as a platform or accepting that constraint knowingly.
- **We have an interest.** The same person maintains AIPLA and the template it derives from. We are not a neutral party recommending ourselves, and this section should be weighed accordingly. The licensing and ownership position of the underlying template would need to be settled before anyone built on it.

Option 3 — parallel systems

KU builds its platform; AIPLA continues as it is. This is a legitimate choice. It is the lowest-coordination path, and it may simply be that KU IT's requirements, procurement position, or existing commitments make the other options moot.

Its cost is duplication in the layers that are institutional rather than disciplinary: spend governance, logging, document handling, single sign-on, and the guidance that has to be written for each of them. If this is the direction, we would like to know early, because it changes what is worth building on the AIPLA side.

What we suggest

Option 1 in the near term, and a conversation about Option 2. The tenancy work is small, useful under every scenario, and would give both sides a concrete integration to learn from before any larger decision is taken. Whether AIPLA is also a useful starting point for the wider platform is a question we would rather explore with your team than assert.

5. Resource list (what UCPH would provide)

Quantities depend on the hosting option chosen (section 4). Figures are for pilot scale (roughly 10 teachers and their classes) and are order-of-magnitude for planning.

Compute (application — modest):

- Container hosting: a Kubernetes namespace **or** 1–2 virtual machines running Docker. Approximately **4 vCPU / 8 GB RAM total** for the web app, backend, and a small document-parsing service.
- A container image registry and a CI runner for builds.

GPU (AI model inference — the main hardware decision; see section 6):

- **Option C (recommended):** one high-end GPU (e.g. **NVIDIA H100 80 GB**) for the main model — comfortably runs a current single-GPU open-weight model such as Qwen 3.6 (27–35B)¹ — plus one smaller GPU (e.g. **L4 / A10, 24 GB**) for text-search embeddings and speech-to-text.
- **Option D (full sovereignty):** a cluster of **8 × H200** to serve a frontier-class open-weight model (e.g. GLM 5.2, ~744B parameters, at FP8 — roughly 1.1 TB aggregate VRAM)², plus the smaller embeddings / speech GPU.
- Inference is served with standard open-source software (vLLM or Ollama).

Database:

- **PostgreSQL 15+ with the pgvector extension** — serves application data, document-search vectors, session state, and analytics. Managed Postgres if available; otherwise a standard (ideally highly-available) instance.

Object storage:

- **S3-compatible storage (e.g. MinIO) or NFS** — uploaded documents, optional lesson recordings, caches. Tens of GB at pilot scale; audio dominates if recording is enabled.

Identity:

- **Teacher single sign-on** from the UCPH identity provider (OpenID Connect or SAML). Students require **no** identity provider (anonymous codes only).

Supporting services:

- A secrets store (e.g. HashiCorp Vault or Kubernetes secrets).
- A logging / metrics stack (Grafana + Prometheus / Loki, or ELK).
- Ingress with TLS certificates; a policy decision on whether any external model API calls are permitted (relevant only for hybrid options B / C).

6. AI models and GPU sizing (the main open decision)

During the pilot the application uses top-tier commercial models for quality. For self-hosting, these are replaced by **open-weight models** run on UCPH GPUs. The model choice is being validated by an ongoing capability evaluation against real physics-tutoring tasks; the **hardware to provision depends on which hosting option (section 4) UCPH chooses**:

¹Qwen 3.6 sizes and VRAM (27B dense and 35B-A3B MoE; ~17–21 GB at 4-bit, fits a single 24 GB GPU): *Qwen 3.6 VRAM & Hardware Requirements*, Will It Run AI (2026) — <https://willitrunai.com/blog/qwen-3-6-vram-requirements>; *Qwen3.6 35B-A3B: Specifications and GPU VRAM*, apxml — <https://apxml.com/models/qwen36-35b-a3b>.

²GLM 5.2 architecture and VRAM (~744B total / 39B active MoE, 1M-token context, MIT-licensed; ~744 GB at FP8, fits one 8 × H200 node): *GLM-5.2 vLLM recipe* — <https://recipes.vllm.ai/zai-org/GLM-5.2>; *Self-Host GLM 5.2: Hardware, vLLM Setup, and Cost*, ofox.ai (2026) — <https://ofox.ai/blog/glm-5-2-self-host-vllm-hardware-cost-2026/>; *GLM-5.2 1M context on a single 8×H200 node*, Phala — <https://phala.com/posts/glm-5-2-1m-context-8xh200>.

Tier	Example open-weight model	Hardware	Suitable for
Single GPU (Option C floor)	Qwen 3.6 (27B dense, or 35B-A3B MoE)	1 × H100 / A100 80 GB (runs at 4-bit on a 24 GB GPU)	Tutor chat, document tasks; the realistic starting point
GPU cluster (Option D)	GLM 5.2 (~744B params / 39B active MoE, 1M context, MIT-licensed — currently the strongest open-weight model)	8 × H200 at FP8 (~1.1 TB aggregate VRAM); BF16 needs more	All tasks including the hardest reasoning; whole-institution scale
Supporting GPU	Embeddings + speech-to-text (Whisper)	1 × 24 GB GPU (or CPU for embeddings)	Document search + voice

We are happy to walk your team through the evaluation results and recommend a specific configuration once your GPU availability is known.

Responsiveness and capacity

Two figures matter, and they answer different questions:

- **Per-student streaming speed** — how fast a single reply appears. Human reading is roughly 5–10 tokens per second, so anything above ~20–30 tokens per second per student already feels immediate. Both recommended setups exceed this comfortably for an individual user.
- **Aggregate throughput** — the total tokens per second served across *all* students at once (the serving software batches requests continuously). This is the capacity figure.

Recent published benchmarks put a single H100 serving a Qwen-3-class model at roughly **2,000+ tokens/second aggregate**³, and an 8 × H200 cluster serving GLM 5.2 at about **1,200 tokens/second**⁴ (with speculative decoding). In practice that means a single H100 comfortably serves a full classroom concurrently, while each student’s reply still streams faster than they can read.

Setup	Aggregate throughput	Per-student feel
Option C — Qwen 3.6 on 1 × H100	~2,000+ tokens/s	tens of tokens/s (well above reading speed)
Option D — GLM 5.2 on 8 × H200 (FP8)	~1,200 tokens/s	~30–60 tokens/s

³Qwen-3-class throughput on a single H100 (~2,352 tokens/s aggregate via vLLM): *Optimizing Qwen3-32B Throughput on NVIDIA H100*, GPUStack — <https://docs.gpustack.ai/2.0/performance-lab/qwen3-32b/h100/>; day-one cost/throughput comparison for Qwen3.6-27B: *Qwen3.6-27B bakeoff*, LLMKube — <https://llmkube.com/blog/qwen3-6-27b-bakeoff>.

⁴GLM 5.2 throughput on 8 × H200 (~1,215 tokens/s sustained output, FP8, SGLang + EAGLE speculative decoding; ~1,370 tokens/s on 8 × B200): *GLM-5-FP8 Inference Benchmark: 8×B200 vs 8×H200* — <https://gist.github.com/BenHamm/dcd09f595fef141567a39582f502cef4>.

Making responses faster (no extra hardware). The largest gains for a tutoring workload come from software, not silicon:

- **Tiered model routing** — most tutoring turns are handled by the smaller, faster model (Qwen 3.6), reserving the large model for genuinely hard reasoning. The typical reply is then both faster and cheaper.
- **Prefix caching** — the tutor’s fixed instructions and lesson context are identical every turn, so they are processed once and cached, sharply reducing the time to the first word of a reply.
- **Speculative decoding** (e.g. EAGLE) — 1.5–3× faster reply streaming; the GLM 5.2 figure above already uses it.
- **FP8 precision** — already assumed in the sizing above.

Newer or additional hardware (e.g. B200) raises both numbers further, but is only needed if concurrency or large-model latency must improve beyond the above.

7. Questions for UCPH IT

The June 2026 version of this document asked whether you could host AIPLA. Since you are already building internal model infrastructure, the useful questions have changed. **Group A is the one we would most like answered**; group B remains relevant if AIPLA is hosted separately from the KU platform.

A. About the KU platform (new — see section 4b)

1. **Inference access.** Will the KU platform expose model inference to other university applications, or only to end users through its own interface? If the former, over what interface — an OpenAI-compatible HTTP API, or something else? What would AIPLA need to do to be admitted as a consumer?
2. **Model catalogue.** Which open-weight models are you planning to serve, at what sizes, and on what hardware? We have measured several candidates against real Danish stx physics tasks and are happy to share the results, which may be useful to you independently of AIPLA (section 8).
3. **Single sign-on.** Which protocol does the UCPH identity provider support for application integration — OpenID Connect or SAML? Can AIPLA register as a client, and what is the process?
4. **Platform scope.** How far up the stack does the KU platform intend to go? Model access and sign-on only, or also spend governance, document corpora, logging, and per-department configuration? This determines whether AIPLA is a tenant, a template, or a parallel system (section 4b), and it is the single answer that most changes what we build next.
5. **Timing.** When do you expect inference and sign-on to be available to other applications, even in a limited form? We would rather integrate early against something incomplete than late against something finished.

B. About hosting AIPLA itself

6. **GPU hosting.** Do you have, or can you provision, GPU hosting for model inference — a single high-end GPU (Option C) or a cluster (Option D)? What specification, and on what timeline? If the KU platform’s GPUs serve this need, questions 6 and 1 are the same question.
7. **Containers and database.** Do you offer Kubernetes or VM hosting, and a managed **PostgreSQL** (ideally with the **pgvector** extension)? Is S3-compatible object storage available?
8. **Data-protection precedent.** Is there an existing pattern for student-facing services that has already cleared your data-protection review, which we could align with rather than starting from scratch?
9. **External connectivity (hybrid only).** For a hybrid setup, is it acceptable for the application to call an **EU-hosted** commercial model API for a small set of the hardest tasks (sending only the prompt, no student identity), or must the deployment be fully self-contained?

8. What we can contribute

- This requirements document and the accompanying migration notes.
- **Infrastructure-as-code (Terraform)** plus operational runbooks, so your team can stand the system up against the resources above — or read them as a costing reference for your own deployment.
- **A capability evaluation of open-weight models on real Danish upper-secondary physics tasks**, refreshed as new models release. As of July 2026 it measures eleven models on exam problem-solving and nine on figure-reading, in five runs each, against published ministry answer keys. Two findings look relevant to the KU model catalogue independently of AIPLA: a single-GPU open model already clears our usefulness threshold on both text and figures, and **no single open model does both well** — a self-hosted tier needs a text model and a vision-language model, routed by whether the question carries a figure. This is also the evidence base for right-sizing model choice per task, which bears directly on both cost and energy use. A standalone summary is available.
- A walkthrough session with your team to answer questions and agree the target configuration.

We are not on a deadline for this. AIPLA’s technical lead is engaged with the project to at least April 2027, and the system continues to run on EU cloud with no disruption in the meantime. Integration or migration can proceed at whatever pace suits your own build. What we would most like in the near term is a conversation about section 4b, early enough to be useful to both sides.